中山大學
SUN YAT-SEN UNIVERSITY

题目分析

M. Water

- 现在有一台饮水机，每一桶水可以提供C升的水。
- 现在有n个人，每个人想要装c_i升的水
- 如果一个人在装水的过程中（包括恰好装完）饮水机没水了，那么他就会离开，同时换上一桶新的水。
- 问（在包括第一桶的情况下）需要多少桶水。

M. Water

- 维护一个剩余水量的变量lst，然后写if

```
int n,C;
scanf("%d%d",&n,&C);
int lst=C,ans=1;
while (n--)
{
    int x;
    scanf("%d",&x);
    lst=lst-x;
    if (lst<=0) {ans++;lst=C;}
}
printf("%d\n",ans);
```

E Catch-Hamster

题意

求 $[0, 10^n)$ 内 3×2^a 的倍数的个数。

E Catch-Hamster

题意

求 $[0, 10^n)$ 内 3×2^a 的倍数的个数。

题解

解法一：推式子。

题目要求的其实就是：$\lceil \frac{10^n}{3 \times 2^a} \rceil$。

注意题干中有一个非常重要的条件：$a \leqslant n$。因此我们可以将上式进一步化为：$\lceil \frac{5^a 10^{n-a}}{3} \rceil$。

我们记 $r = 3 - (5^a 10^{n-a} \bmod 3)$，由向上取整的定义可得：

$ans = \lceil \frac{5^a 10^{n-a}}{3} \rceil \equiv \frac{5^a 10^{n-a}+r}{3} \bmod 998244353$。

通过费马小定理和快速幂即可求出 ans。每次求解的复杂度为 $O(\log n)$。

E Catch-Hamster

在p为质数，且b不是p的倍数时，有

$$\frac{a}{b} \equiv a * b^{p-2} (mod\ p)$$

因为$b^{p-1} \equiv 1 (mod\ p)$

$$a/b * b \equiv a (mod\ p)$$

$$a * b^{p-2} * b \equiv a * 1 \equiv a (mod\ p)$$

因此，$\frac{a}{b} \equiv a * b^{p-2} (mod\ p)$

H. coprime

- 问有多少个长度为n的序列满足

1. 每一个元素均在$[1, m]$
2. 相邻的元素互质

H. coprime

- 考虑DP
- f[i][j]表示前i个位置，最后一个位置是j的方案数
- 转移为
- $f[i][j] = \sum f[i-1][k](\gcd(j,k) = 1)$
- 直接暴力枚举k可能有点卡，可以记录下对于j有哪些符合的k直接转移。
- 也可以引入μ函数做到更好的复杂度。

H. coprime

- 但比较奇怪的是，看到有许多队伍都预处理了符合的k直接转移。
- 但实现比较好的0.5s就通过了此题
- 有的队伍在时限开到2s依然TLE
- 然后最后决定将时限开到了5s
- 这里说一下可能加快代码效率的点：
- 1.两个小数的加法取模可以通过减法来实现
- 即c=(a+b)%mod，可以表示成c=a+b;if (c>mod) c-=mod;
- 2.尽可能多用int

J. intervals

- 现在给你一个长度为n的序列
- 定义一个区间是好的，当且仅当它包含有至少有一个奇数和至少一个偶数
- 多组询问，每一次询问一个区间$[X, Y]$
- 问该区间里有多少个子区间是好的

J. intervals

- 将所有的奇数看作1，偶数看作0
- 相邻的奇数看成一段，相邻的偶数看成一段
- 如果一个区间在一段内，则是不好的，否则则是好的
- 容斥，用全部减去不和法的
- 记第i段有w_i个
- 答案即为$C_n^2 - \sum C_w^2$

J. intervals



- 多组询问统计一个类似前缀和的答案，然后再补上两端杯分割的段。

K. Xor

- 定义一个长n的排列p价值为 $\sum_{i=1}^{n} p_i \oplus i$，其中 $\oplus$ 表示异或。求价值最大的长度为n的排列。

K. Xor

- 我们定义一个函数solve(n)，表示[1,n]的最优匹配方案
- 有solve(1)=[1]
- 对于任意一个n，找到一个最高的$2^i \leq n$，我们尽可能使得i这一位不“浪费”
- 具体来说，我们让
- 2^i匹配$2^i - 1$, $2^i + 1$匹配$2^i - 2$，如此类推
- 如果右侧不够，那么左侧剩下一个前缀，可以递归解决子问题
- 如果左侧不够，右侧在忽略若干最高位后依然是一个前缀，可以递归解决子问题

C Catch-Hamster

题意

一棵n(1 ≤ n ≤ 10^5) 个点的树，上面有若干仓鼠，仓鼠不会走同一条边两次除非到了一个叶子节点。你可以在节点上放陷阱，你需要使用最少的陷阱使得无论它们最初的位置和逃跑策略如何，你都能抓住所有仓鼠

C Catch-Hamster

问题等价于任意两个叶子节点的路径上都必须要有一个陷阱，这是充要的

我们可以使用动态规划或者贪心解决这个问题

首先我们选择一个非叶子节点作为根，特判掉n≤2的答案

我们设dp[i][0/1]表示以i为节点的子树，0没有/1有一个叶子节点可以通过节点i走向i的父亲并且路径上没有陷阱，我们显然不允许这样的节点超过1个

考虑是否在i节点放置陷阱即可完成转移

C Catch-Hamster

对于贪心法而言，本质上是如果某个节点只有在必须放陷阱时我们才放陷阱

如果对于子树i而言，设叶子节点可以通过节点i走向i的父亲并且路径上没有陷阱的数量为k

如果k=0，显然没有必要放陷阱；

如果k>1，显然必须要放陷阱；

如果k=1，在i的父亲放置陷阱和在i节点放置陷阱同样合法，并且在i的父亲放置陷阱可以拦截更多的仓鼠，因此i处没有必要放陷阱

A Forest

题意

一个n(6 ≤ n ≤ 10^5) 个点无重边无自环的无向连通图，有k(0 ≤ k < n) 个三度点，其余点度小于3，构造两个满足如上要求的图，使得边数最多和最少，多组数据。

A Forest

考虑边数最多怎么做?

边数最多时，顶点的度数和越大，也就是构造出的图k个点的度数为3，其余n-k个点的度数尽量为2（最小化1度点个数）

这里给出一种构造方法。首先我们将n个点连成一个环，此时所有节点度数均为2

如果k为偶数，我们只需要将这k个点两两连一条边，此时没有1度点，度数和最大

如果k为奇数，此时必须要有一个1度点，由于k<n，我们只需在k个点外任选一个点，断掉这个点的两条边后连向k个点中没有配对的那个点即可

A Forest

同样的，边数最少时，顶点的度数和越小，也就是构造出的图k个点的度数为3，其余n-k个点的度数尽量为1

首先我们将所有3度点连成一条链

当k+2≤n-k时，我们用多的点将3度点的度数补够，剩下的随便连成连通图即可，此时是一棵树，边数显然最少

当k+2>n-k时，剩余的度数为m=(k+2)-(n-k)，如果m是偶数，那么直接3度点之间两两连即可，此时所有k个点外的点都是1度点，显然边数最少；如果m为奇数，此时必须有一个2度点，由于k<n，我们只需在k个点外任选一个点，连向一个3度点将m变为偶数即可

F. Fighting in Group



题意

n 个人，编号 1 到 n。每个人都想和与自己**编号不相邻**的**所有人**“切磋”至少一遍。且他们都**不希望**与自己编号相邻的人“切磋”。

每一轮时，你可以安排两队人相互切磋。对于 a 和 b，只要某一轮中他们都在场且处在不同的队伍里，就认为 a 和 b 彼此切磋过。

问你至少要安排多少轮才能满足这些人的切磋要求。输出方案。

F. Fighting in Group

根据题意，任何时候都应该满足x与x+1不能分在两队

首先我们考虑第一步应该怎么做，假设有n个人，此时任意两个人都没有打过。

我们应该这样选择分配队伍最优：

依次选定k个断点，设为$a_1, a_2, \ldots, a_k$，

得到$[1, a_1 - 1], [a_1 + 1, a_2 - 1], \ldots, [a_k + 1, n]$这些区间，

把编号是奇数区间分到一队，偶数的分到一队

F. Fighting in Group

设$f(n)$表示n个人最优解，我们先假设$k=1$，假设断点设在$a \in [2, n-1]$，

我们把$[1, a-1]$放一队，把$[a+1, n]$放一队，

此时打完之后，剩下的就是要处理$[1, a]$与$[a, n]$两个相同的子问题

两个子问题除了a重复了之外，其他是互不影响的，

因此这两个问题只需要一轮就可以变成4个子问题

可以推出式子$f(n) = \min_a(\max(f(a), f(n-a)) + 1)$，

而$f(n)$是关于n单调不降的，

所以令$a = \lceil n/2 \rceil$最优，因此$f(n) = f(\lceil n/2 \rceil) + 1$

如果$k=2$时，则$f(n) = f(\lceil n/3 \rceil) + 2$，是没有$k=1$优的，显然更大的$k$不需要考虑

F. Fighting in Group



例子

上面讲的可能比较抽象，这里举一个 $n = 9$ 的例子吧:

第一轮: 待解决 $[1, 9]$。

方案: $\{1, 2, 3, 4\}$ vs $\{6, 7, 8, 9\}$ ($\{5\}$)。

第二轮: 待解决 $[1, 5], [5, 9]$。

方案: $\{4, 5, 6\}$ vs $\{1, 2, 8, 9\}$ ($\{3, 7\}$)。

第三轮: 待解决 $[1, 3], [3, 5], [5, 7], [7, 9]$。

方案: $\{1, 5, 9\}$ vs $\{3, 7\}$ ($\{2, 4, 6, 8\}$)。

分治结束，构造完成，共 3 轮。

G. MicrosoftHearts



- 两个人在玩红心大战
- 规则如下：
- 先手出一张牌
- 如果后手有相同花色的牌，则必须跟相同花色的牌。此时谁的点数大则可以获得两张牌，同时获得下一次出牌的先手
- 如果后手没有，则可以任意出。先手获得两张牌，先后手不变
- 最后的得分是红心牌的数量，得分小的获胜
- 问先手是否必胜

G. MicrosoftHearts

- 对抗搜索
- $F[x][y][0/1]$表示双方手牌剩余状态为x，y，现在是谁先手，Alice最优收益
- 状态数估算是$2\sum_{i=1}^{n} C_n^{i\,2}$，实际并没有这么大
- 运用Alpha-Beta 剪枝

B. a math problem

- 定义函数$f(x)$表示有多少个数对 (p,q)， 满足

1. gcd(p,q)=1
2. p，q均是x的约数。

- 令$g(n) = \sum_{i=1}^{n} f(i)$
- 求$g(n)$

B. a math problem

- 定义函数$f(x)$表示有多少个数对 (p,q)， 满足

1. gcd(p,q)=1
2. p，q均是x的约数。

- 考虑$f(x)$怎么求
- 令$x = \prod p_i^{a_i}$
- $f(x) = \prod (2a_i + 1)$
- 直接min_25筛处理

B. a math problem

- 定义函数$f(x)$表示有多少个数对 (p,q)， 满足

1. gcd(p,q)=1
2. p，q均是x的约数。

$$
\begin{aligned}
ans &= \sum_{p=1}^{n}\sum_{q=1}^{n/p}[gcd(p,q)=1]\frac{n}{pq} \\
&= \sum_{T=1}^{n}\frac{n}{T}\sum_{d|T}[gcd(d,\frac{T}{d})=1] \\
&= \sum_{T=1}^{n}\frac{n}{T}\sum_{d|T}\mu^2(d) \\
&= \sum_{d=1}^{n}\mu^2(d)\sum_{i=1}^{\frac{n}{d}}\frac{n}{id}
\end{aligned}
$$

$$
\sum_{i=1}^{n}\mu^2(i)=\sum_{i=1}^{\sqrt{n}}\frac{n}{i^2}\mu(i)
$$

D. Coffee

- 有n天，第i天的咖啡价格是a_i
- 你有m张优惠卷，第i张可以减免w_i，并且将于r_i天之后过期
- 一天只可以用一张优惠卷
- 问恰好喝k杯咖啡需要最少需要多少钱

D. Coffee

- 记买 i 杯咖啡的最小费用为 $f(i)$，可以证明 $f(i)$ 是下凸的
- 考虑wqs二分，二分一个权值，每杯咖啡减去这个值，问题变为任意买使得费用最小
- 任意买的答案可以用一个堆来维护
- 倒着往前扫，如果一张优惠卷可以用了就加入队列，如果当前的价格小于0则购买。同时将该咖啡也加入优惠卷队列，表示“换杯”

L. FYC’s luv letter



题意

本题给出一个初始字符串，一个由 n 个字符串构成的字典，开始时，我们的字符串在字典的某个位置，题目定义了三种魔法操作，分别是将初始字符串两边删除一个字符，增加一个字符，以及从字典的某一个字符串跳跃到另一个字符串，并给出了各种操作所需的花费。
要求将初始字符串变化成字典上另一个字符串，并跳跃到目标字符串所在的位置。题目要求我们给出最小的花费。

L. FYC’s luv letter



解法

不难想到，定义两个不同字符串之间的距离为魔法操作的花费，然后将每个子回文串抽象成一个点，我们就可以将问题转化为图上的最短路径问题。

由于字符串全程都是回文串，而一个字符串中包含的本质不同的子回文串数量是 $O(n)$ 级别的，所以图中的点和边也是 $O(n)$ 级别的。

而操作数并不多，最后的图边数也不会太大，可以用任何一种较快的单源最短路算法解决（没卡 spfa）。

L. FYC's luv letter



解法

难点在于建图。
建图的关键在于找出字典中的所有本质不同的字符串，然后将它们映射到图上。这里有多种方法解决。
想到这里，我们发现最终建好的图像极了一个自动机，没错！我们可以直接使用回文自动机，然后把建出来的回文树作为最终的图，跑单源最短路。
当然，只要能处理出所有本质不同的字符串，然后把它们映射出来建图，就可以解决这道问题，比如 manacher 算法也可以解决这个问题。
本题给出了 manacher 和 PAM 两种算法的 STD。时间复杂度均为 $O(n\log n)$ 。并且 manacher 的效率和代码难度均低于 PAM。